

Structuri de Date — Sintează completă: B-Trees (Arbori B)

Sintează după cursul AC233D_1. Conține tot ce e pe slide-uri + completări valoroase (cazurile de ștergere, conexiuni) + corecții ale unor erori de transcriere din slide-uri + legături directe cu întrebările de examen.

Legendă marcate:

- [EXAMEN] = apare direct într-un model de examen pe care l-am analizat.
- [CAPCANĂ] = greșeală tipică / formulare-capcană.
- [CORECTIE] = slide-ul are o eroare; varianta corectă e cea de aici.

0. De ce există B-Trees? Motivația (stocarea secundară)

Tot ce am studiat până acum (BST, AVL, heap, hash) presupune că **structura încapă complet în memoria principală (RAM)**. În acel model:

- toate operațiile de bază costă la fel;
- chiar și asta e o aproximare grosieră — nici măcar RAM-ul nu e „plat”, din cauza ierarhiei de cache-uri.

Dar unele aplicații (baze de date, sisteme de fișiere) au **mai multe date decât încapă în RAM** → trebuie folosite structuri care stau pe **disc (secondary storage)**.

Fapt-cheie de reținut:

Discul este de 10.000–100.000× mai lent decât RAM-ul.

Consecința de design e fundamentală și e „de ce”-ul întregului capitol:

Costul dominant nu mai e numărul de operații CPU, ci **numărul de accesări la disc** (Disk-Read / Disk-Write). Vrem o structură care **minimizează numărul de noduri vizitate**, fiindcă fiecare nod = o accesare la disc.

1. Ideea centrală: noduri „grase” (multe chei într-un nod)

- Într-un **arbore binar echilibrat**, $\boxed{n}$ chei → înălțime $\boxed{h = \lceil \log_2 n \rceil}$, iar fiecare operație importantă accesează $\boxed{O(h)}$ noduri.
- **Ideea B-tree:** pune **mai multe chei + pointeri la copii într-un singur nod**. Creștem gradul (numit *degree* sau *branching factor*) la $\boxed{d > 2}$, deci înălțimea scade la $\boxed{h = \lceil \log_d n \rceil}$.

- În practică (d) poate fi de ordinul **miliar** — un nod e fix cât un **bloc de disc**, ca să-l citești dintr-o singură accesare.

Exemplu sugestiv (din curs):

Dacă ($d = 1000$), atunci doar **3 accesări** ($h = 2$) acoperă **un miliard de chei** (fiindcă $1000^3 = 10^9$).

Asta e toată magia: schimbi baza logaritmului din 2 într-un număr mare → arborele devine foarte **plat**, deci foarte puține accesări la disc.

2. Definiția formală a unui B-Tree

2.1. Câmpurile unui nod (x)

Fiecare nod (x) are:

- ($x.n$) = numărul de chei stocate în nod;
- ($x.key[1] \leq x.key[2] \leq \dots \leq x.key[x.n]$) = cele ($x.n$) chei, în **ordine nedescrescătoare**;
- ($x.leaf$) = flag boolean: ($true$) dacă (x) e frunză, ($false$) dacă e nod intern;
- ($x.c[1], x.c[2], \dots, x.c[x.n + 1]$) = cei ($x.n + 1$) **pointeri** la copii (doar dacă (x) e nod intern).

Observație esențială: un nod cu ($x.n$) chei are exact ($x.n + 1$) **copii** — cu unul mai mult decât numărul de chei. (E generalizarea directă a BST-ului: la BST un nod cu 1 cheie are 2 copii.)

2.2. Proprietatea de ordine (cheile delimitează intervale)

Cheile din nod împart spațiul în ($x.n + 1$) intervale, fiecare „acoperit” de un copil:

Pointer	Subarborele conține cheile...
$x.c[1]$	$k \leq x.key[1]$
$x.c[2]$	$x.key[1] \leq k \leq x.key[2]$
$x.c[3]$	$x.key[2] \leq k \leq x.key[3]$
...	...
$x.c[x.n+1]$	$k \geq x.key[x.n]$

E **generalizarea proprietății BST**: în loc de „stânga < nod < dreapta”, ai $(x.n)$ chei care separă $(x.n + 1)$ zone.

2.3. Proprietățile structurale (definiția completă)

1. **Toate frunzele au aceeași adâncime.** (Arborele e perfect echilibrat pe înălțime — asta e ce-l ține $O(\log n)$ în worst-case, fără rotații.)
2. Fie $(t \geq 2)$ *gradul minim (minimum degree)* al arborelui:
 - Orice nod **altul decât rădăcina** are **cel puțin $(t - 1)$ chei** (deci cel puțin (t) copii).
 - Orice nod are **cel mult $(2t - 1)$ chei** (deci cel mult $(2t)$ copii).
 - Un nod e **PLIN (full)** când are **exact $(2t - 1)$ chei**.
 - Un nod plin are **$(2t)$ copii**.
3. **Rădăcina e excepția**: poate avea de la **1 cheie** (minimul) până la $(2t - 1)$. (Adică nu e obligată să respecte limita inferioară $(t - 1)$.)

Tabel de reținut (limitele pe nod)

	Min chei	Max chei	Min copii	Max copii
Nod non-rădăcină	$(t - 1)$	$(2t - 1)$	(t)	$(2t)$
Rădăcină (≥ 1 cheie)	(1)	$(2t - 1)$	(2)	$(2t)$

[EXAMEN] [CAPCANĂ] Întrebare directă din grila 2023 (Q21): „Un B-tree de grad minim (n) — orice nod altul decât rădăcina conține cel mult ___ chei?” → Răspuns: $(2n - 1)$ (ei notează gradul minim cu (n) în loc de (t)). Capcana e că opțiunile mai conțin $(2n)$, $(n-1)/2$ etc. — limita corectă e $(2t - 1)$, NU $(2t)$. ($(2t)$ e numărul de **copii**, nu de chei.)

Conexiune utilă: Un B-tree cu $(t = 2)$ se numește **arbore 2-3-4** (fiecare nod are 2, 3 sau 4 copii). Arborii 2-3-4 sunt „variante de bază” a arborilor Red-Black — un Red-Black tree e, în esență, reprezentarea binară a unui arbore 2-3-4. Bun de știut pentru întrebările care leagă structurile între ele.

2.4. Exemplu concret de B-tree (din curs)

Rădăcina $([25, 72])$, cu trei copii:

- $([10, 15]) \rightarrow$ frunzele $(6, 9) \mid (13, 14) \mid (19, 24)$
- $([30, 54, 65]) \rightarrow$ frunzele $(27, 29) \mid (40, 44) \mid (61, 62) \mid (70, 71)$
- $([94]) \rightarrow$ frunzele $(83, 91) \mid (96, 99)$

Verificare: cheile $\boxed{25}$ și $\boxed{72}$ din rădăcină separă intervalele $\boxed{<25}$, $\boxed{25-72}$, $\boxed{>72}$. Toate frunzele sunt la aceeași adâncime. ✓

3. Înălțimea unui B-Tree (teoremă + demonstrație)

Teoremă: pentru un B-tree cu $\boxed{n \geq 1}$ chei și grad minim $\boxed{t \geq 2}$:
$$h \leq \log_t \frac{n+1}{2}$$

Demonstrație (numărăm minimul de noduri pe fiecare nivel):

Cel mai „înalt” (cel mai prost) B-tree e cel cu cât mai puține chei per nod. Numărul **minim** de noduri pe nivele:

- Nivel 0 (rădăcina): $\boxed{1}$ nod, cu $\boxed{\geq 1}$ cheie.
- Nivel 1: $\boxed{\geq 2}$ noduri (rădăcina are minim 2 copii).
- Nivel 2: $\boxed{\geq 2t}$ noduri.
- Nivel 3: $\boxed{\geq 2t^2}$ noduri.
- ...
- Nivel $\boxed{h}$: $\boxed{\geq 2t^{(h-1)}}$ noduri.

Fiecare nod non-rădăcină are $\boxed{\geq t - 1}$ chei. Deci numărul total de chei:

$$n \geq 1 + (t - 1) \sum_{i=1}^h 2t^{i-1} = 1 + 2(t - 1) \cdot \frac{t^h - 1}{t - 1} = 1 + 2(t^h - 1) = 2t^h - 1$$

De aici: $\boxed{2t^h - 1 \leq n} \Rightarrow \boxed{t^h \leq (n+1)/2} \Rightarrow \boxed{h \leq \log_t((n+1)/2)}$. ■

Interpretare: înălțimea e logaritmică în bază $\boxed{t}$. Cu $\boxed{t}$ mare, baza e mare $\rightarrow$ arborele e foarte plat $\rightarrow$ foarte puține accesări la disc. Exact poanta din Secțiunea 1.

4. Căutarea (Search)

```
B-Tree-Search(x, k)
1  i = 1
2  while i ≤ x.n and k > x.key[i]
3      i = i + 1
4  if i ≤ x.n and k == x.key[i]
5      return (x, i)
6  if x.leaf
```

```

7     return nil
8 else Disk-Read(x.c[i])
9     return B-Tree-Search(x.c[i], k)

```

Cum funcționează:

1. În fiecare nod, scanezi **liniar** cheile cât timp $(k > x.key[i])$ (te oprești la prima cheie $(\geq k)$).
2. Dacă ai dat de egalitate $(k == x.key[i]) \rightarrow$ ai găsit, întorci $(nod, index)$.
3. Altfel, dacă ești în **frunză** $\rightarrow nil$ (cheia nu există).
4. Altfel, cobori în copilul $(x.c[i])$ (mai întâi **Disk-Read** ca să-l aduci de pe disc) și repeți.

Complexitate:

- **Accesări la disc:** $O(h) = O(\log_t n)$ — câte un **Disk-Read** per nivel.
- **CPU:** în fiecare nod scanezi $O(t)$ chei liniar, pe (h) nivele $\rightarrow O(t \cdot \log_t n) = O(t \cdot h)$.
 - *Notă:* dacă folosești **căutare binară** în interiorul nodului, CPU-ul scade la $O(\log t \cdot \log_t n) = O(\log n)$. Slide-ul folosește varianta liniară.

5. Inserarea (Insertion)

5.1. Principiul: inserezi mereu într-o frunză, cu „split proactiv” pe o singură coborâre

- O cheie nouă se inserează **întotdeauna într-o frunză**.
- Strategia e „**single pass down**”: cobori o singură dată spre frunza potrivită și, pe drum, **splituiești orice nod plin** $(2t - 1)$ chei) **înainte** de a coborî în el.
- De ce proactiv? Ca să garantezi că, atunci când vrei să împingi o cheie-mediană în sus în părinte, **părintele nu e niciodată plin** (are loc). Așa eviți „cascadarea” spliturilor în sus.

5.2. Splitting — divizarea unui copil plin

Când copilul (y) în care vrei să cobori e plin $(2t - 1)$ chei), îl **splituiești**:

- **cheia-mediană** (a (t) -a cheie, $(y.key[t])$) **urcă** în părintele (x) ;
- (y) păstrează primele $(t - 1)$ chei;
- un **nod nou** (z) primește ultimele $(t - 1)$ chei;
- părintele (x) capătă o cheie și un pointer în plus.

Exemplu (din curs): copilul plin $y = [14, 35, 42, 51, 55, 68, 70]$ are 7 chei $\Rightarrow 2t - 1 = 7 \Rightarrow t = 4$. După split:

- mediană = a 4-a cheie = **51** urcă în părinte;
- $y = [14, 35, 42]$ (primele $t-1 = 3$);
- $z = [55, 68, 70]$ (ultimele $t-1 = 3$);
- părintele devine $[... 10, 51, 85 ...]$.

```

B-Tree-Split-Child(x, i, y)                                // y = x.c[i] este plin (2t-1 chei)
1  z = Allocate-Node()
2  z.leaf = y.leaf
3  z.n = t - 1
4  for j = 1 to t - 1
5      z.key[j] = y.key[j + t]                                // ultimele t-1 chei -> z
6  if not y.leaf
7      for j = 1 to t
8          z.c[j] = y.c[j + t]                                // ultimii t copii -> z
9  y.n = t - 1                                                // y pastreaza primele t-1 chei
10 for j = x.n + 1 downto i + 1
11     x.c[j + 1] = x.c[j]                                    // fac loc pentru noul pointer
12 for j = x.n downto i
13     x.key[j + 1] = x.key[j]                                // fac loc pentru cheia mediana
14 x.key[i] = y.key[t]                                        // mediana urca in x
15 x.n = x.n + 1
16 Disk-Write(y)
17 Disk-Write(z)
18 Disk-Write(x)

```

[CORECȚIE] Pe a doua copie a acestui pseudocod din slide-uri apar două greșeli de transcriere: linia 5 e scrisă `x.key[j] = x.key[j + t]` și linia 6 `if not x.leaf`. **Corect** este `z.key[j] = y.key[j + t]` și `if not y.leaf` (varianta de pe slide-ul „Splitting” e cea bună, redată mai sus).

Complexitate split (întrebare pusă în curs, cu răspuns):

$\boxed{t}$ operații CPU (copiezi $\boxed{t}$ chei și pointeri) + **exact $\boxed{3}$ operații Disk-Write** (scrii $\boxed{y}$, $\boxed{z}$, $\boxed{x}$).

5.3. Inserarea sub un nod care nu e plin

```
B-Tree-Insert-Nonfull(x, k)
1 i = x.n // presupunem x NU e plin
2 if x.leaf
```

```

3   while i ≥ 1 and k < x.key[i]
4       x.key[i + 1] = x.key[i] // deplasez cheile mai mari la dreapta
5       i = i - 1
6   x.key[i + 1] = k           // inserez k pe pozitia corecta
7   x.n = x.n + 1
8   Disk-Write(x)
9   else while i ≥ 1 and k < x.key[i]
10      i = i - 1
11      i = i + 1
12      Disk-Read(x.c[i])
13      if x.c[i].n == 2t - 1    // copilul in care cobor e plin
14          B-Tree-Split-Child(x, i, x.c[i])
15          if k > x.key[i]      // dupa split, decid in care jumatate
cobor
16              i = i + 1
17      B-Tree-Insert-Nonfull(x.c[i], k)

```

- **(x) e frunză:** deplasezi cheile mai mari la dreapta și pui **(k)** pe locul lui.
- **(x) e intern:** găsești copilul potrivit; dacă e plin, îl splituești (acum părintele are loc), apoi cobori recursiv.

5.4. Procedura principală de inserare

```

B-Tree-Insert(T, k)
1   r = T.root
2   if r.n == 2t - 1           // radacina e plina
3       s = Allocate-Node()    // creez o radacina noua, goala
4       T.root = s
5       s.leaf = false
6       s.n = 0
7       s.c[1] = r             // vechea radacina devine copilul ei
8       B-Tree-Split-Child(s, 1, r) // o splituiesc
9       B-Tree-Insert-Nonfull(s, k)
10  else B-Tree-Insert-Nonfull(r, k)

```

Cel mai important fapt despre înălțime: singura situație în care un B-tree **crește în înălțime** e când **rădăcina e plină** — atunci se creează o rădăcină nouă deasupra și se splituie cea veche. Deci **un B-tree crește DOAR de sus, la rădăcină** (spre deosebire de BST, care crește la frunze). Asta e și motivul pentru care toate frunzele rămân la aceeași adâncime.

Exemplu (din curs): nodul plin $[35, 42, 51, 55, 68]$ are 5 chei $\Rightarrow 2t-1 = 5 \Rightarrow t = 3$. Rădăcina se splituie: $\text{root} = [51]$, $y = [35, 42]$, $z = [55, 68]$.

5.5. Complexitatea inserării

- CPU: $O(t \cdot h) = O(t \cdot \log_t n)$
- Accesări la disc: $O(h) = O(\log_t n)$

Alegerea lui t optim depinde de:

- raportul dintre viteza CPU/RAM și timpul de acces la disc;
- **block-size**-ul discului (cât de mare poate fi un obiect citit/scriș dintr-o singură accesare).

6. Ștergerea (Deletion)

Slide-ul spune doar „*Not hard, but more tedious*” + o figură, fără pseudocod. Completez cu cazurile standard (CLRS), fiindcă **ștergerea e relevantă la examen** (vezi întrebările despre succesor / poziția cheilor).

Problema principală: spre deosebire de inserare (care lovește doar frunze), ștergerea poate viza **chei aflate și în noduri interne**, nu doar în frunze.

[CORECȚIE] [CAPCANĂ] Slide-ul scrie textual: „*Insertion: merging nodes. Deletion: splitting nodes.*” — **acest rând este inversat (transpus)**. Varianta corectă, standard:

- **Inserarea SPLITUIEȘTE noduri** (când un nod se umple, $2t-1$ chei, îl divizezi).
- **Ștergerea ÎMBINĂ (MERGE) noduri** (când un nod scade sub $t-1$ chei) sau împrumută chei de la frați.

Reține **varianta corectă** pentru examen: *insert* → *split*, *delete* → *merge*. (E intuitiv: inserarea adaugă, deci „crapă”; ștergerea scoate, deci „lipește”.)

Invariantul de menținut: orice nod non-rădăcină trebuie să păstreze $\geq t - 1$ chei.

Simetric inserării, strategia e **proactivă**: pe drumul în jos te asiguri că nodul în care urmează să cobori are $\geq t$ chei (cu una mai mult decât minimul), ca să poți șterge din el fără să încalci invariantul.

Cazurile de ștergere a cheii k (rezumat CLRS)

Caz 1 — k e într-o FRUNZĂ: o ștergi direct (frunza are oricum $\geq t$ chei datorită strategiei proactive).

Caz 2 — k e într-un nod INTERN x :

- **2a.** Copilul y dinaintea lui k (cel care conține **predecesorul**) are $\geq t$ chei → înlocuiești k cu predecesorul k' , apoi ștergi recursiv k' din y .

- **2b.** Copilul z de după k (cel cu **succesorul**) are $\geq t$ chei $\rightarrow$ înlocuiești k cu succesorul k' , apoi ștergi recursiv k' din z .
- **2c.** Ambii copii (y și z) au exact $t-1$ chei $\rightarrow$ **merge** $y + k + z$ într-un singur nod ($2t-1$ chei), apoi ștergi k din nodul îmbinat.

Caz 3 — k NU e în nodul curent x (trebuie să cobori în copilul $c = x.c[i]$): dacă c are doar $t-1$ chei, întâi îl „îngrași”:

- **3a.** Un frate adiacent al lui c are $\geq t$ chei $\rightarrow$ **împrumuți** o cheie prin părinte (o rotație: o cheie din părinte coboară în c , o cheie din frate urcă în părinte).
- **3b.** Ambii frați au $t-1$ chei $\rightarrow$ **merge** c cu un frate (coborând și o cheie din părinte).

Complexitate ștergere:

- **Accesări la disc:** $O(h) = O(\log_t n)$
- **CPU:** $O(t \cdot h) = O(t \cdot \log_t n)$

(Aceleași ordine de mărime ca inserarea.)

7. Cheat-sheet final — fapte de reținut pentru examen

Toate astea apar sau pot apărea în grile. Le memorezi „la sânge”:

1. **Chei/nod (non-rădăcină):** $t - 1 \leq \#chei \leq 2t - 1$. **Copii:** $t \leq \#copii \leq 2t$.
Rădăcina: $1 \dots 2t-1$ chei. $\rightarrow$ [EXAMEN, grila 2023 Q21] non-rădăcină are cel mult $2t-1$ chei (la ei $2n-1$).
2. **Cea mai mare cheie** dintr-un B-tree (cu mai mult de un nod) e **întotdeauna într-o frunză** — și **nicăieri altundeva**. Motiv: în orice nod intern, copilul cel mai din dreapta conține chei mai mari decât cheile nodului, deci cheile unui nod intern nu pot fi maximul global; maximul global = ultima cheie din frunza cea mai din dreapta. $\rightarrow$ [EXAMEN, Exam1-2025 Q22] răspuns: „într-o frunză, dar nu altundeva”.
3. **Succesorul** unei chei x poate fi în **trei locuri** (deci „toate variantele”):
 - în **același nod** — dacă x e într-o frunză și **nu** e ultima cheie a frunzei (succesorul = cheia următoare din aceeași frunză);
 - într-un **descendent** — dacă x e într-un **nod intern** (succesorul = minimul subarborelui din dreapta lui x , adică o frunză de sub el);
 - într-un **strămoș** — dacă x e **ultima cheie a unei frunze** (urci până dai de prima cheie mai mare). $\rightarrow$ [EXAMEN, Model Seria 13 Q21] răspuns: „și în același nod, și într-un strămoș, și într-un descendent”.
4. **Toate frunzele sunt la aceeași adâncime** (echilibrare perfectă pe înălțime, fără rotații).

5. **B-tree-ul crește în înălțime DOAR la rădăcină** (prin split de rădăcină). Crește „de sus”, nu „de jos” ca BST-ul.
6. **Insert → split. Delete → merge.** (NU invers! — vezi corecția de la Secțiunea 6.)
7. **Înălțimea:** $h \leq \log_t((n+1)/2)$ — logaritmică în bază t .
8. **Complexități** (atât insert, cât și delete, cât și search):
 - accesări disc: $O(\log_t n)$;
 - CPU: $O(t \cdot \log_t n)$.
 - Split: $O(t)$ CPU + **3 Disk-Write**.
9. **De ce B-trees?** Discul e de $10^4-10^5 \times$ mai lent ca RAM → minimizezi **accesările la disc** mărinđ gradul nodului (un nod = un bloc de disc). Cu $d = 1000$, 3 accesări acoperă 10^9 chei.
10. $t = 2 \Leftrightarrow$ **arbore 2-3-4** $\Leftrightarrow$ „scheletul” arborilor Red-Black.